UNIDAD 3: VERIFICACIÓN Y VALIDACIÓN (V&V) CON HERRAMIENTAS

BLOQUE 2: TESTING PRÁCTICO Y CONTRATOS (SEMANA 2)

1. IMPLEMENTACIÓN AVANZADA DE PRUEBAS UNITARIAS Y ASEVERACIONES DE GRADO INDUSTRIAL

La escritura de pruebas unitarias en el laboratorio profesional de software no se limita a verificar caminos felices o flujos estándar. Exige la validación sistemática de condiciones de éxito bajo estrés lógico y el control estricto de casos de borde o excepciones lógicas. La estructura fundamental de cualquier prueba unitaria automatizada sigue el patrón de las tres etapas de la ingeniería de pruebas, comúnmente denominado patrón AAA: Configuración del escenario (Arrange), Ejecución de la lógica (Act) y Verificación de los resultados mediante aseveraciones (Assert).

1.1 El rol crítico de las Aseveraciones (Asserts) en la lógica de control

Una aseveración es una expresión lógica o booleana que evalúa si el resultado obtenido dinámicamente por el software coincide exactamente con el valor teórico esperado por el plano de diseño. Si la condición de la aseveración se evalúa como verdadera, el test continúa su ejecución; si se evalúa como falsa, el framework de pruebas interrumpe inmediatamente el hilo de ejecución del caso, registra el fallo en la memoria del sistema y genera un reporte detallando el desvío (indicando el valor esperado frente al valor real obtenido) junto con el rastreo de la pila de llamadas (stack trace) para localizar la línea exacta del error.

Existen diferentes tipos de aseveraciones avanzadas que se deben dominar en el laboratorio:

- **Aseveraciones de igualdad y estado:** Evalúan la correspondencia exacta entre tipos de datos primitivos u objetos complejos.
- **Aseveraciones de identidad y referencia:** Verifican si dos variables apuntan exactamente al mismo espacio de memoria física.

- **Aseveraciones de colecciones:** Inspeccionan de forma exhaustiva si un arreglo o lista contiene elementos específicos, si respeta un ordenamiento determinado o si su longitud final concuerda con la lógica de negocio.

1.2 Validación de Casos de Borde e Inyección de Errores Controlados

El software de alta confiabilidad debe validar de forma mandatoria los límites extremos de sus variables operativas. En el código de producción, esto implica verificar comportamientos ante valores nulos (null), punteros vacíos, cadenas de texto de longitud cero o que superen los límites de almacenamiento, desbordamientos numéricos (overflow) o entradas fuera de rango.

Las pruebas unitarias deben forzar deliberadamente estos escenarios mediante la inyección de datos inválidos. El objetivo de ingeniería en este punto es inspeccionar que los mecanismos de manejo de excepciones del sistema (bloques try/catch o interceptores de errores) capturen la anomalía, limpien los recursos de memoria y respondan lanzando una excepción controlada con un mensaje técnico claro, en lugar de permitir que el hilo de ejecución colapse y provoque la caída abrupta de la aplicación en el entorno del usuario.

2. ARQUITECTURA DE AISLAMIENTO: IMPLEMENTACIÓN DE DOBLES DE PRUEBA (MOCKS Y STUBS)

En un ecosistema de desarrollo moderno basado en componentes aislados o microservicios, las funciones de negocio interactúan constantemente con dependencias de infraestructura como motores de bases de datos, sistemas de archivos, colas de mensajería o APIs de proveedores externos. Si una prueba unitaria realiza una llamada de red o escribe en una base de datos real, deja de ser una prueba unitaria y se transforma en una prueba de integración pesada. Esto introduce lentitud en el laboratorio, dependencia del estado del entorno y alta inestabilidad. Para garantizar el aislamiento atómico absoluto del componente bajo examen, la ingeniería de software recurre al diseño de **Dobles de Prueba (Test Doubles)**.

2.1 Stubs: Simuladores de Estado e Inyección de Datos Indirectos

Un Stub es un objeto simulado de estructura estática que reemplaza a una dependencia real para proporcionar respuestas preconfiguradas o "enlatadas" ante las llamadas de métodos que realiza el componente bajo prueba. El Stub carece de lógica interna compleja; simplemente intercepta la petición del componente y devuelve inmediatamente el conjunto de datos

necesario para que la función bajo examen continúe su camino de ejecución. Su propósito fundamental es simular estados de la infraestructura y proveer entradas indirectas al componente, permitiendo probar cómo reacciona el código ante diferentes respuestas de una base de datos o API externa sin interactuar físicamente con ellas.

2.2 Mocks: Verificadores de Comportamiento y Auditoría de Interacciones

Un Mock es un doble de prueba dinámico y considerablemente más avanzado que un Stub. Su enfoque no se limita a proveer datos mínimos, sino que está diseñado específicamente para auditar e inspeccionar el comportamiento y el protocolo de interacción del componente bajo examen. El Mock registra activamente cada una de las interacciones que recibe: qué métodos exactos invocó el componente, con qué parámetros o argumentos técnicos específicos se realizaron las llamadas, en qué orden secuencial ocurrieron y cuántas veces se ejecutaron.

Al finalizar la fase de ejecución (Act), la aseveración final de la prueba unitaria no evalúa el estado del componente, sino que interroga directamente al Mock para validar si la lógica de producción interactuó con sus dependencias externas respetando de forma estricta las reglas de comunicación fijadas por la arquitectura del sistema.

3. MÉTRICAS DE CONTROL DE CALIDAD: INGENIERÍA DE COBERTURA DE CÓDIGO (CODE COVERAGE)

La cobertura de código no representa una métrica estética o una simple declaración de intenciones; es un indicador de control auditable que mide el grado de rigurosidad y exhaustividad con el que la suite de pruebas automatizadas ha inspeccionado el código fuente de producción. Las herramientas de cobertura inyectan agentes durante el proceso de ejecución de los tests dinámicos, analizan el árbol de sintaxis abstracta de la aplicación y calculan matemáticamente la densidad de revisión basándose en dos criterios principales de cobertura.

3.1 Statement Coverage (Cobertura de Sentencias o Líneas)

Esta métrica calcula la proporción matemática entre el número de líneas de código fuente ejecutables que fueron recorridas al menos una vez por alguna prueba automatizada y el total de líneas ejecutables del proyecto. Si bien alcanzar una cobertura de sentencias elevada (por ejemplo, mayor al 80%) es un estándar industrial deseable, es un error metodológico asumir

que esto garantiza la ausencia de bugs. Una línea de código puede ejecutarse de forma exitosa con un conjunto de datos específico pero fallar catastróficamente ante un caso de borde que no fue contemplado por la suite de pruebas.

3.2 Branch Coverage (Cobertura de Ramas o Caminos Condicionales)

Es un criterio de control significativamente más riguroso y seguro para el laboratorio. El Branch Coverage no evalúa líneas físicas, sino que analiza todos los caminos condicionales y bifurcaciones lógicas posibles dentro de las estructuras de control del software (como sentencias if, else, operadores ternarios o bloques switch).

Si una función posee un condicional doble, la cobertura de ramas exige de forma mandatoria que la suite de tests contenga al menos un caso que fuerce a la condición a evaluarse como verdadera, y otro caso independiente que la fuerce a evaluarse como falsa. De esta manera, se certifica que todas las rutas lógicas del flujo de control han sido recorridas y validadas, reduciendo drásticamente la presencia de código muerto o comportamientos ocultos en producción.

4. PRUEBAS DE CONTRATO (CONSUMER-DRIVEN CONTRACTS) EN ENTORNOS DISTRIBUIDOS

En arquitecturas basadas en servicios desacoplados o integraciones orientadas a APIs, las pruebas de integración tradicionales que requieren levantar múltiples servidores simultáneamente se vuelven complejas de acoplar y mantener en el espacio de trabajo local de un desarrollador. El **Contract Testing** resuelve esta problemática permitiendo validar de forma aislada las interfaces de comunicación y los contratos de intercambio de datos entre dos componentes independientes, asegurando la compatibilidad del sistema sin incurrir en costos operativos de infraestructura.

4.1 Delimitación Formal de Roles: Provider y Consumer

- **Consumer (Consumidor):** Representa al componente de software, aplicación cliente, servicio frontend o microservicio dependiente que inicia una petición de red y requiere consumir de forma obligatoria los datos estructurados provistos por otro servicio para poder ejecutar su propia lógica de negocio de manera correcta.
- **Provider (Proveedor):** Representa al servicio lógico, microservicio de backend o API que expone un punto de acceso (endpoint) público y se compromete formalmente a

procesar las peticiones del consumidor y entregar respuestas consistentes bajo un conjunto de reglas técnicas predefinidas.

4.2 El Mecanismo del Contrato Técnico Local

El contrato técnico es un archivo de especificación estricto e inmutable (comúnmente modelado mediante un esquema JSON o archivos de especificación OpenAPI) consensuado previamente entre los equipos de desarrollo. Este documento digital define con precisión absoluta las reglas de juego de la interfaz: los nombres técnicos exactos de los campos de datos, las estructuras de anidamiento, los tipos de datos primitivos obligatorios para cada variable (strings, integers, booleans), los parámetros requeridos en la cabecera (headers) y los códigos de estado HTTP válidos ante diferentes escenarios operativos.

Al ejecutarse de manera automatizada en el entorno local de desarrollo, el framework de pruebas confronta tanto al consumidor como al proveedor contra este archivo de contrato de forma independiente. Si el equipo de backend modifica una variable o elimina un campo esperado por el cliente, la prueba de contrato falla de forma inmediata en el flujo local, deteniendo la integración antes de que el cambio incompatible rompa la interfaz de la aplicación en producción.

5. MITIGACIÓN DE ERRORES: REGRESIÓN AUTOMATIZADA LOCAL Y SMOKE TESTS

5.1 Pruebas de Regresión Automatizadas en el Espacio de Trabajo Local

Cada vez que un desarrollador introduce un cambio en el código fuente para implementar una nueva funcionalidad, resolver una deuda técnica de diseño o corregir un defecto preexistente, introduce de forma colateral un riesgo matemático elevado de alterar de manera impredecible el comportamiento de otros componentes del sistema que ya funcionaban correctamente.

Las pruebas de regresión consisten en la reejecución sistemática, estricta y automatizada de la suite completa de pruebas lógicas ante cualquier modificación detectada en el repositorio local. Esta práctica actúa como una red de seguridad técnica que garantiza al desarrollador que el nuevo incremento de valor no ha degradado la estabilidad global del sistema ni ha reintroducido fallos del pasado.

5.2 Diseño e Implementación de un Smoke Test de Salud Funcional

A diferencia de las suites de regresión exhaustivas, un Smoke Test (Prueba de Humo) es una colección ultra rápida y acotada de pruebas automatizadas diseñada específicamente para certificar que los componentes mínimos vitales e indispensables de la aplicación se encuentran operativos inmediatamente después de levantar el entorno local de desarrollo o aplicar un cambio estructural en el entorno de pruebas. Su diseño básico para un microservicio de backend consiste en validar de forma automatizada tres estados de salud críticos:

1. **Liveness:** Verificar que el proceso principal del servidor ha iniciado de forma correcta y se mantiene activo en la memoria del sistema operativo sin cerrarse inesperadamente.
2. **Readiness:** Comprobar que los canales de comunicación de red, sockets o puertos locales asignados responden adecuadamente a las peticiones de prueba.
3. **Healthcheck:** Interrogar al endpoint base de salud del microservicio para certificar que la aplicación devuelve de forma unificada un código de éxito técnico (HTTP 200 OK) y confirma que sus conexiones internas mínimas se encuentran estables.

6. METODOLOGÍAS DE LABORATORIO EN EL FLUJO DE TRABAJO: TDD Y BDD PRAGMÁTICO

6.1 Test Driven Development (TDD) y su Ciclo Incremental Lógico

El desarrollo guiado por pruebas es una disciplina de ingeniería de software que invierte el flujo tradicional de programación, forzando al desarrollador a diseñar los criterios de aceptación técnicos y los mecanismos de control antes de escribir la solución funcional de producción. Su ejecución se rige por un ciclo iterativo y estrictamente incremental compuesto por tres fases inmutables:

- **Fase Roja (RED):** El desarrollador escribe una prueba unitaria detallada para un requerimiento técnico o funcionalidad que aún no ha sido implementada en el sistema. Al ejecutar la prueba, esta debe fallar obligatoriamente de forma controlada. Este paso es fundamental para certificar que el test es válido, que evalúa la lógica correcta y que está libre de falsos positivos.
- **Fase Verde (GREEN):** El desarrollador escribe la solución de código fuente de producción mínima, más simple y estrictamente necesaria para lograr que la prueba que acaba de fallar pase a estado exitoso. En esta fase no se prioriza la optimización ni la elegancia del código, sino la efectividad de la solución lógica.
- **Fase de Refactorización (REFACTOR):** Con la certeza absoluta de contar con la red de seguridad provista por el test en verde, el desarrollador procede a limpiar la sintaxis del

código implementado, eliminar duplicaciones, optimizar algoritmos y mejorar la legibilidad y el diseño arquitectónico del componente, garantizando que el sistema permanece estable tras cada micro-optimización.

6.2 Behavior Driven Development (BDD) y la Sintaxis Semántica Gherkin

El desarrollo guiado por comportamiento evoluciona las prácticas de TDD, desplazando el foco desde la validación de la estructura técnica interna del código hacia la validación de la lógica del negocio y el comportamiento esperado del sistema desde la perspectiva operacional del usuario. En el laboratorio de desarrollo, los requerimientos y escenarios funcionales se describen de manera formal utilizando un lenguaje ubicuo y semántico estructurado bajo tres cláusulas inmutables:

- **Dado (Given):** Establece el contexto inicial, las precondiciones inmutables y el estado inicial del sistema antes de que ocurra la interacción (ej. "Dado que el usuario tiene una sesión activa y posee un saldo de mil pesos").
- **Cuando (When):** Describe la acción técnica concreta, el evento operativo o la transacción ejecutada por el actor del sistema (ej. "Cuando intenta transferir quinientos pesos al destinatario seleccionado").
- **Entonces (Then):** Especifica la consecuencia lógica directa, la mutación de estado observable en el sistema o la respuesta visible que debe validar de forma obligatoria la aseveración de la prueba (ej. "Entonces el sistema debe descontar el dinero, emitir un comprobante y actualizar el saldo disponible a quinientos pesos").